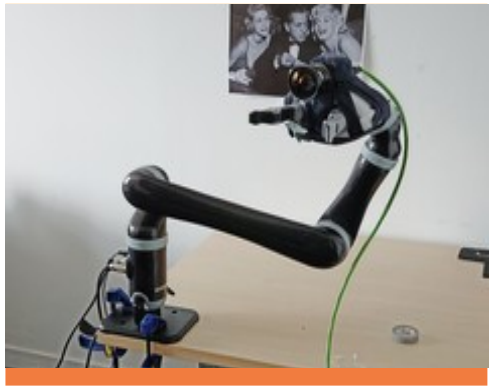
SLAM
VA56 – Introduction to Mobile robotics





UTBM - FISE INFO - VA56
Jocelyn BUISSON

utbm
université de technologie
Belfort-Montbéliard

SLAM

Why SLAM?

- Localization often requires a map
  - Practical issues:
    - Creating a map by hand is time-consuming, error prone
    - Updating an existing map to reflect changes in the environment is also time-consuming, error prone
  - Fundamental issue (circular dependency)
    - The robot cannot have a map without knowing where it is
    - It cannot know where it is without having a map

    Chicken-and-egg problem

SLAM

SLAM Taxonomy

- SLAM systems can be categorized in several ways:
  - Filter-based SLAM
    - States include robot pose and landmarks
    - Updates with prediction and correction steps (Bayesian update)

      Examples: EKF-SLAM, UKF-SLAM, FastSLAM
      - Simple, real-time on small maps
      - $O(N^2)$ scaling on landmarks (not suitable for large-scale environments)

SLAM

SLAM Taxonomy

- SLAM systems can be categorized in several ways:
  - Smoothing / Graph-based SLAM
    - Keeps history of poses & constraints in a factor graph
    - Solves a global optimization problem

      Used by almost all modern SLAM system
      - Can handle large-scale environments
      - Loop-closures naturally integrated
      - Computationally expensive (requires marginalization strategies)

SLAM

SLAM Taxonomy

- SLAM systems can be categorized in several ways:
  - 2D / 3D SLAM
  - Lidar SLAM / Visual SLAM / Multi-sensor SLAM

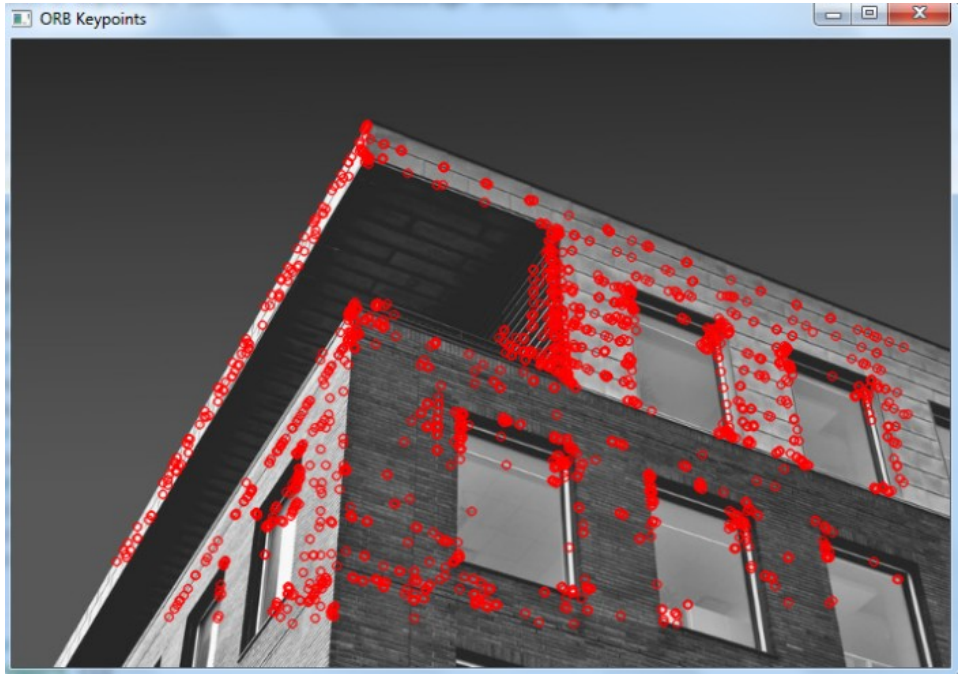
  - Sparse, semi-dense, dense Visual SLAM
    - Sparse: uses only keypoints (ORB-SLAM)
    - Semi-dense: uses edges and high-radient regions (LSD-SLAM)
    - Dense: uses the whole image (DTAM, KinectFusion)

SLAM

SLAM Taxonomy

- SLAM systems can be categorized in several ways:
  - Feature-based Visual SLAM
    - Extracts keypoints from images (ORB, SIFT, FAST-BRIEF, etc.)
    - Match features between frames

      Used in ORB-SLAM, PTAM
      - Robust to lighting changes
      - Requires texture



SLAM

SLAM Taxonomy

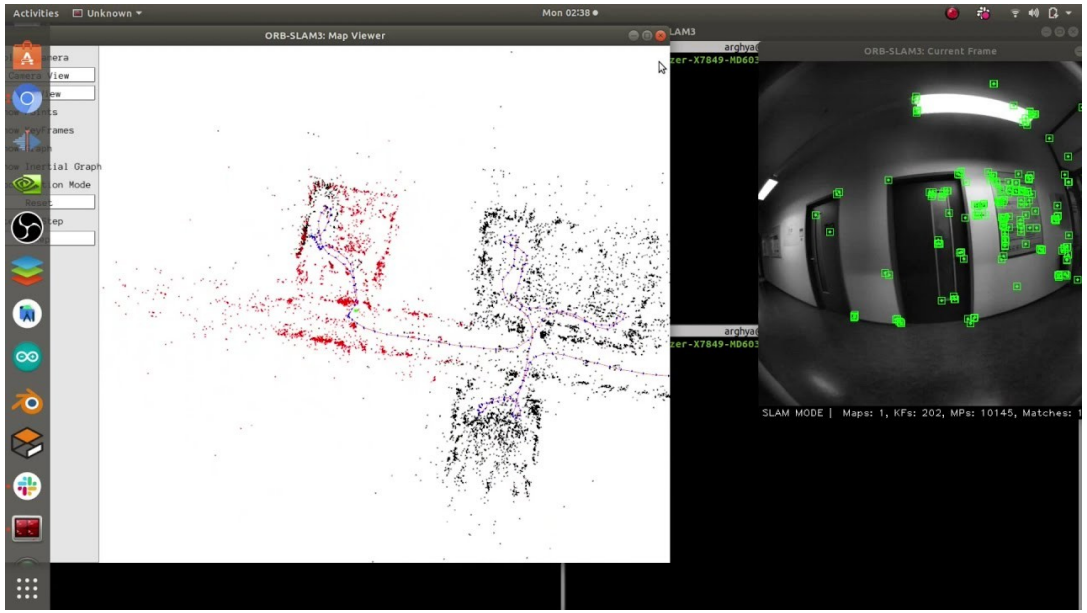
- SLAM systems can be categorized in several ways:
  - Direct Visual SLAM
    - Uses pixel intensities directly (photometric error)

      Examples: DSO, LSD-SLAM
      - Work in low-texture areas
      - Sensitive to illumination changes

SLAM

Modern SLAM systems for ROS

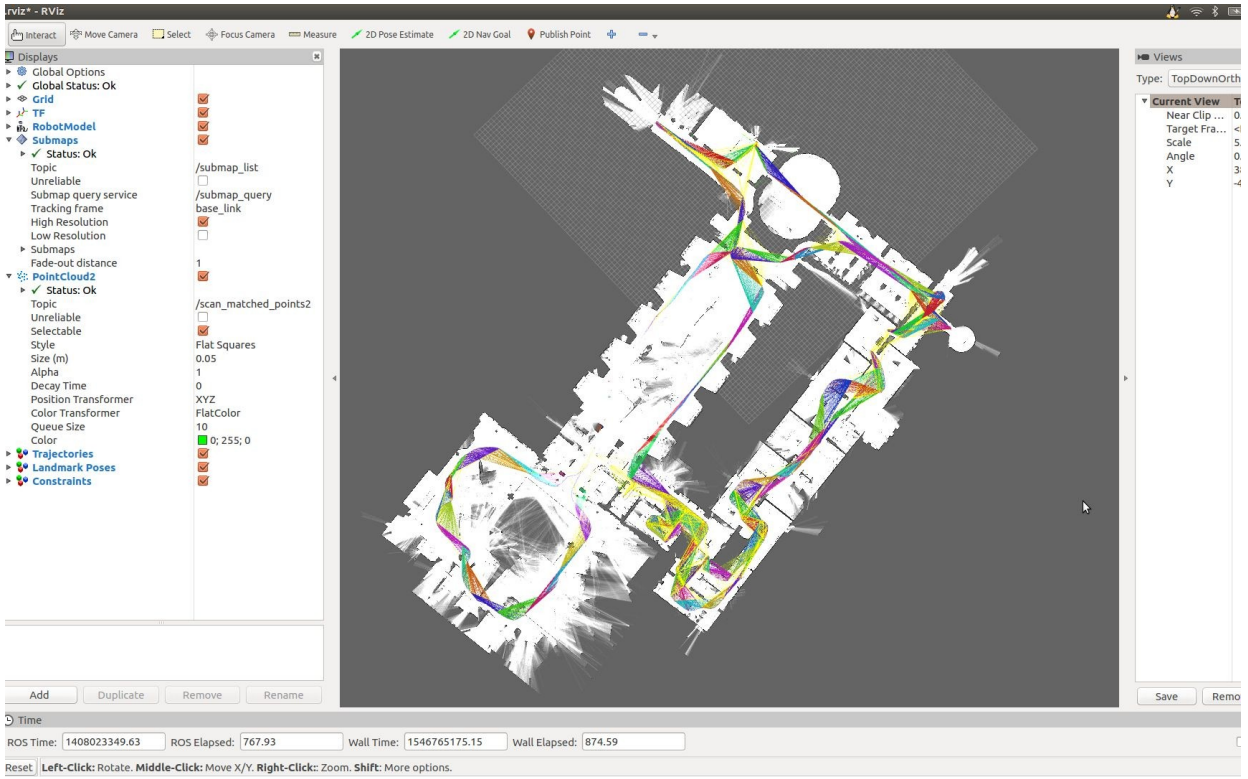
- ORB-SLAM3
  - Feature-based visual SLAM
  - Supports monocular, stereo, RGB-D, IMU fusion
  - Accurate and robust
  - Official ROS1 support, unofficial ROS2 port



SLAM

Modern SLAM systems for ROS

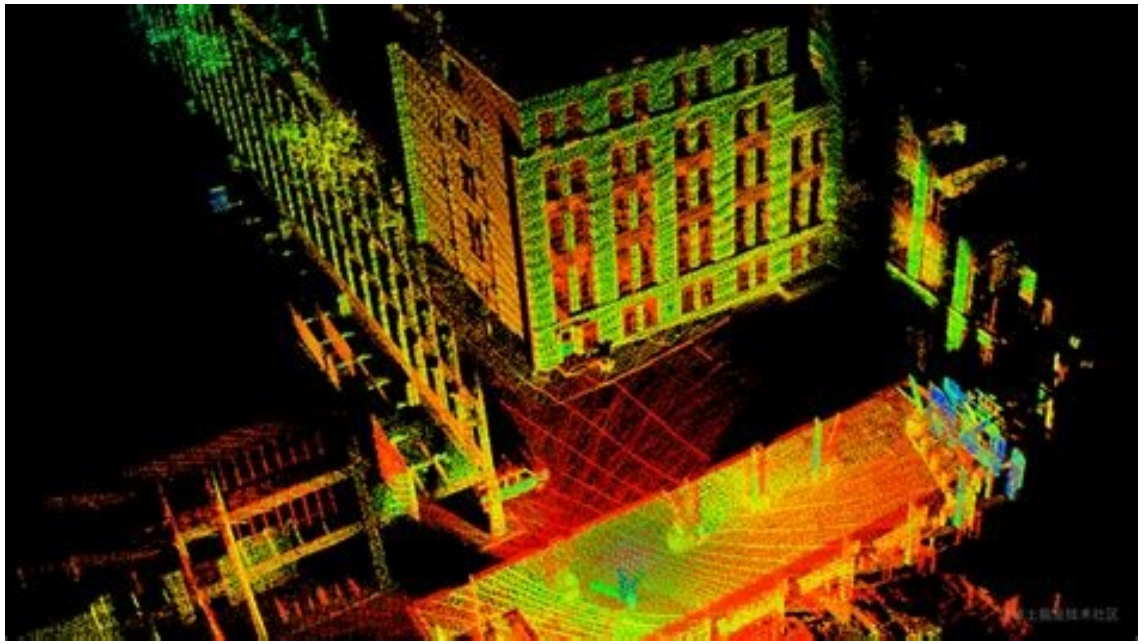
- Cartographer
  - 2D Lidar SLAM
  - ICP / correlation / pose graph based
  - ROS2 package cartographer_ros
  - Very good for indoor buildings



SLAM

Modern SLAM systems for ROS

- LIO-SAM
  - 3D Lidar+IMU fusion
  - IMU preintegration / lidar scan features matching / graph-based
  - Very accurate for 3D mapping
  - ROS2 community port



SLAM

What is SLAM?

- Stands for Simultaneous Localization and Mapping
  - The robot builds a map of an unknown environment
  - The robot localizes itself within the map
  - Both tasks are performed at the same time (often in real-time)
- SLAM is composed of several key components
  - Sensor data acquisition
  - Motion estimation (odometry)

    IMU, wheel odometry, etc.
  - Data association

    Matches current sensor observation to previously seen features

SLAM

What is SLAM?

- SLAM is composed of several key components
  - State estimation

    Kalman filters, particle filters, graph optimization
  - Mapping

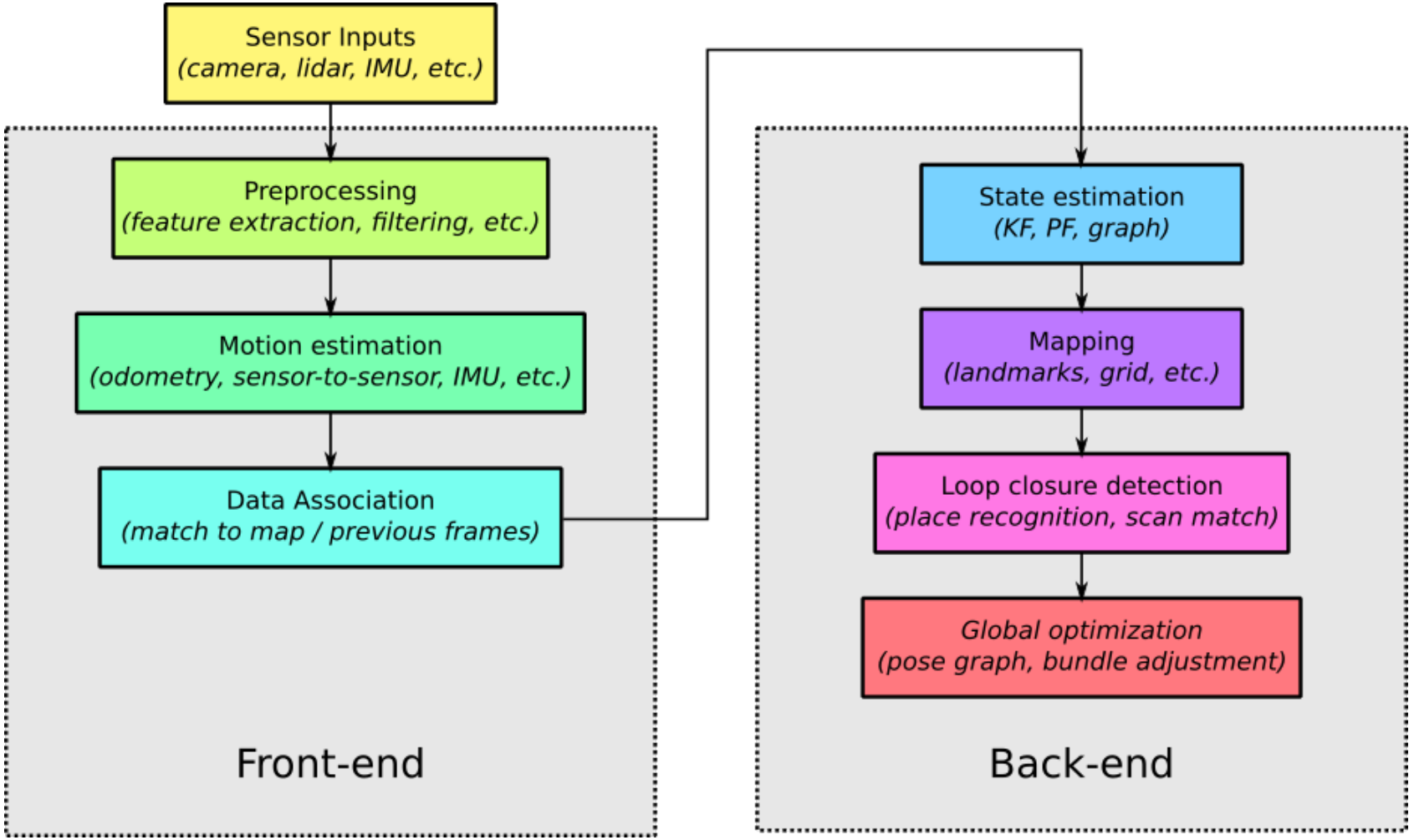
    Builds a representation of the environment (landmarks, point clouds, occupancy grids, mesh, etc.)
  - Loop closure detection

    Detects when the robot returns to a previously visited place
  - Map / Trajectory optimization

    After loop closure, optimizes the whole map and trajectory to reduce distortion/drift over long runs (e.g., bundle adjustment)

SLAM

Typical SLAM pipeline



SLAM

Motion estimation (reminder)

- Estimate motion from model and control inputs

  Differential drive, Ackerman, non-holonomic model, etc.
- Fuse IMU and wheel odometry

  Kalman filtering, preintegration, etc.
- Sensor-to-sensor matching

  Visual odometry (PnP, registration), scan matching (ICP, NDT)
- Or a combination of the above

  Common in modern SLAM systems
- Error accumulates quickly => drift

  SLAM must correct it using loop closures and global optimization

SLAM

Motion estimation (reminder)

- Motion estimation provides a good initial guess for SLAM optimization

  Bundle adjustment, pose graph optimization, etc.

- Mathematically:

$$x_k = f(x_{k-1}, u_k^{\text{eff}}) + w_k \quad \text{or} \quad p(x_k | x_{k-1}, u_k^{\text{eff}})$$

Where u_k^{eff} is the effective motion input, i.e., either the control inputs or relative motion measured by a sensor

SLAM

Data association

- Match current sensor observations to previously seen landmarks or features

  Have we seen this before, where is it in the map?

$$\hat{m}_j = \arg\max_j p(z_k^i | x_k, m_j)$$

  $\hat{m}_j$ is used to update the map in the state estimation step

- Challenges (similar to sensor-to-sensor motion estimation)
  - Sensor noise (uncertainty)
  - Features can look similar (ambiguity)
  - Environments can be dynamic (moving objects)
  - Wrong association (large errors in map and pose estimation)

SLAM

Data association

- Approaches
  - Nearest neighbor (geometric matching)
  - Visual features descriptors (ORB, SIFT, SURF)
  - Point cloud or laser scan map matching (ICP, NDT)
- Ensures correct map updates
- Critical for loop closure and drift correction

SLAM

State estimation (reminder)

- Estimate the robot's pose (and optionally landmark states) over time, given:
  - Motion estimates (odometry, sensor-to-sensor)
  - Current sensor observations
- Fuse predictions and measurements to reduce uncertainty

  Bayesian filtering

$$p(x_k|z_{1:k}, u_{1:k}) = \eta\, p(z_k|x_k) \int p(x_k|x_{k-1}, u_k)\, p(x_{k-1}|z_{1:k-1}, u_{1:k-1})\, dx_{k-1}$$

  - Kalman Filter
  - Particle Filter
  - Graph-based optimization
- Provides a posterior estimate of robot pose and map
- Corrects drift from motion estimation

SLAM

Graph-based optimization

- Solve optimization problems by representing variables and constraints as a graph
  - Nodes are the variables to estimate

    $$X = \{x_1, \ldots, x_n\}$$

    Poses, positions, twists, etc.
  - Edges are constraints between variables

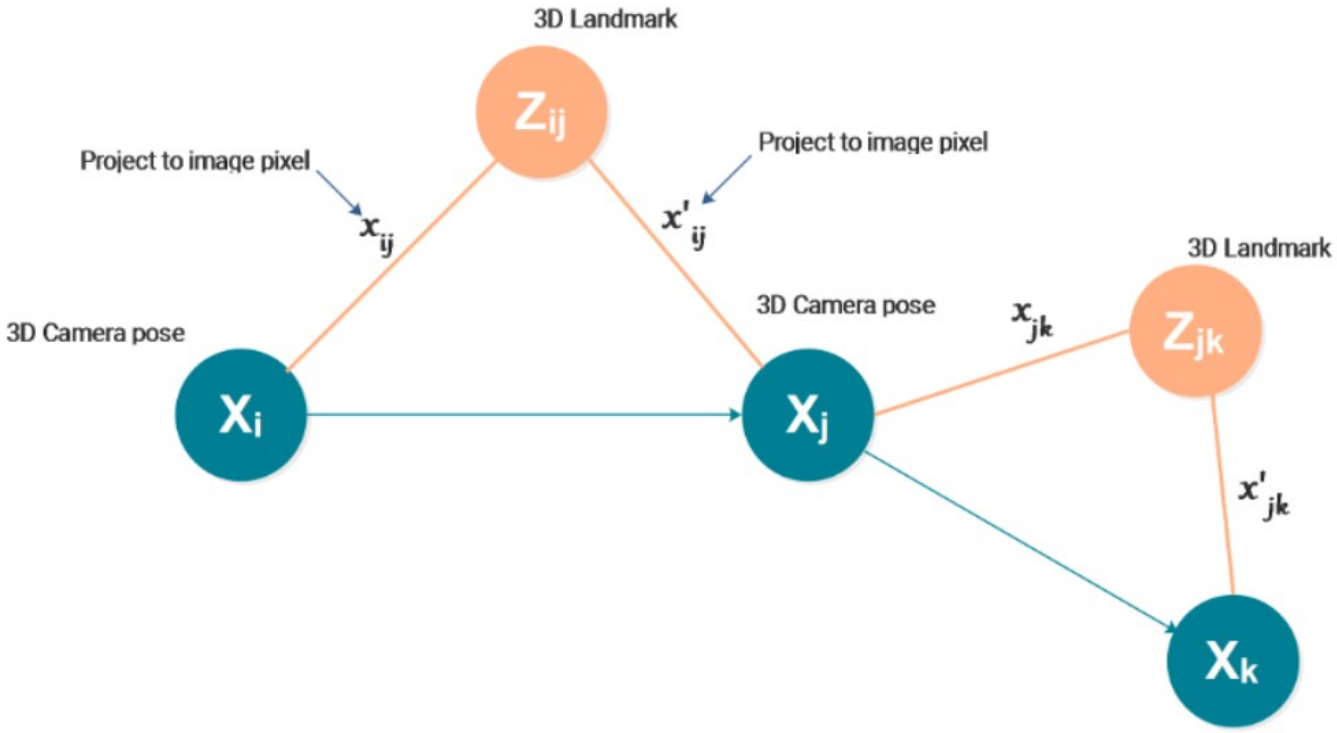
    Represents observations, measurement relationship
    - Residual / error $e_{ij}(x_i, x_j)$
    - Uncertainty Σ_{ij}
- Mathematical form

$$X^* = \arg\min_X \sum_{(i,j)\in \mathrm{E}} ||e_{ij}(x_i, x_j)||^2_{\Sigma_{ij}}$$

Can be solved using nonlinear iterative solvers (e.g., Gauss-Newton, Levenberg-Marquardt)

SLAM

Graph-based optimization



SLAM

Mapping

- Build a representation of the environment using current observations and estimated robot poses

  Landmarks, occupancy grid, point clouds, mesh, etc.
- Mathematically

$$m \leftarrow \text{update}(m, x_k, z_k)$$

  - Map is updated incrementally with each new pose and observation
  - Use current pose to transform observations into map frame
  - Fuse new information with existing map knowledge
  - Exact update depends on map representation

SLAM

Mapping

- Map representations
  - Landmark-based maps (2D/3D point landmarks)
  - Occupancy grid maps
    - Cells store probability of occupancy
    - Allow path planning
    - Mostly 2D but can be extended to 3D for small-scale environments
  - Octomap
    - Tree-based 3D occupancy grid (Octree)
    - Memory efficient (adaptive resolution)
  - Point cloud maps
    - Simple, dense, easy to build
    - Large memory, requires data structure to query

SLAM

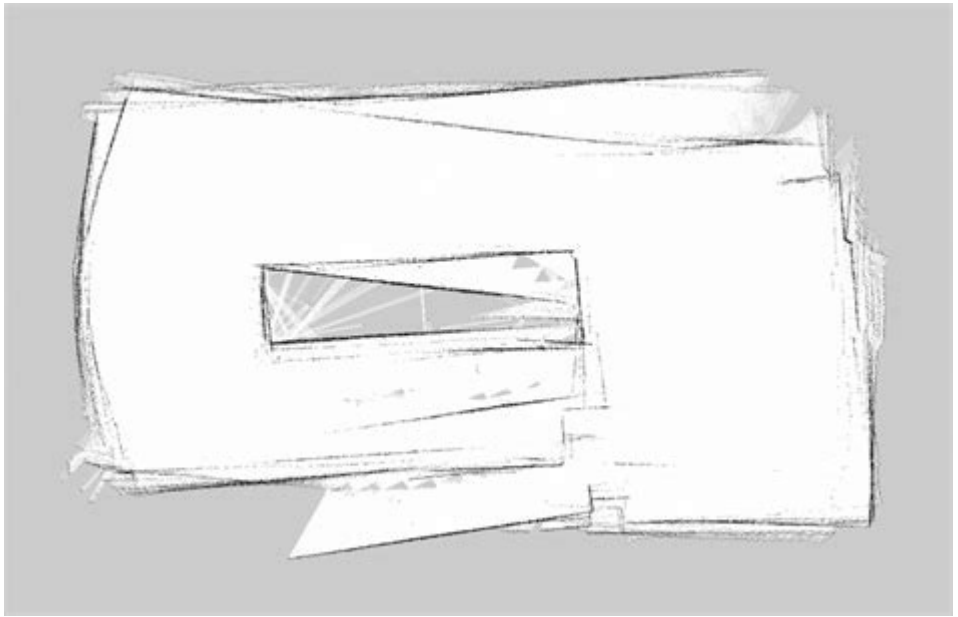
Mapping

- Map representations
  - Mesh maps (triangulation derived from point clouds)
  - Signed distance field volumetric maps
  - Deep learning:
    - Semantic maps
    - Neural implicit maps (Nerf, Gaussian splatting)

SLAM

Mapping

- Relies on correct data association
- Errors in motion estimation accumulate in the map
- Loop closure provide global constraints to correct map drift



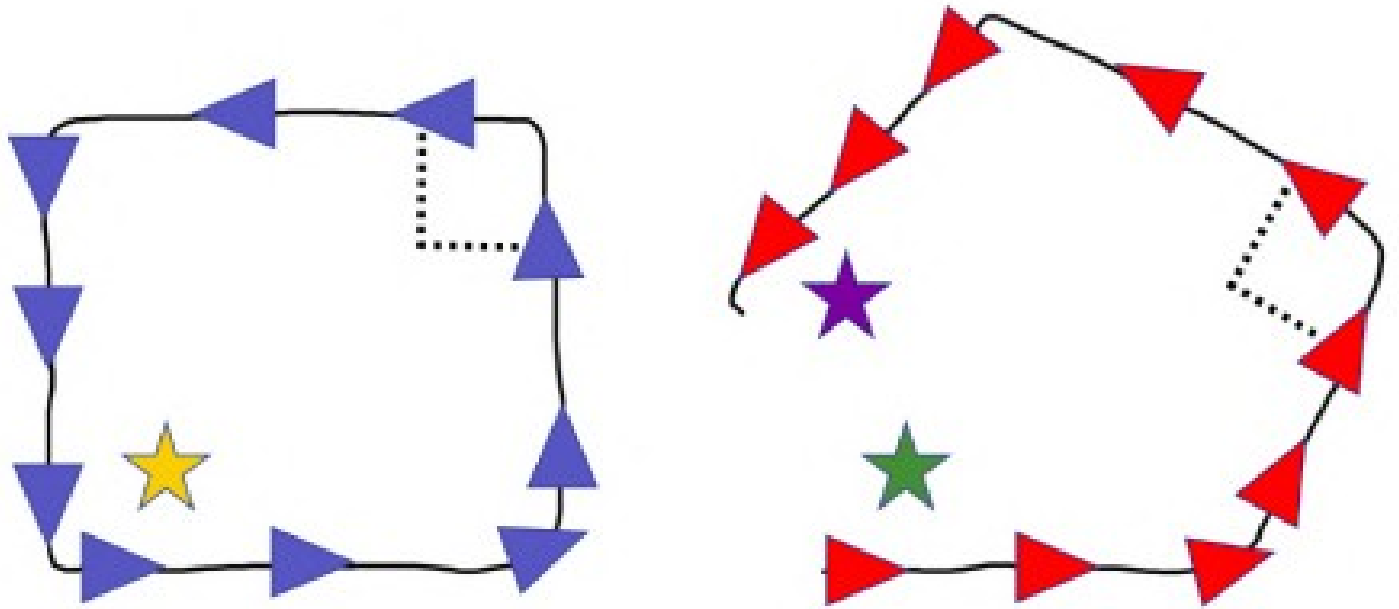
SLAM

Loop closure & Map / Trajectory optimization

- Detect when the robot revisits a previously seen place
  - Corrects drift accumulated from motion estimation
  - Ensures global consistency of the map and trajectory
- Common approaches
  - Visual SLAM: Bag-of-Words, ORB descriptors, etc.;
  - Lidar SLAM: scan-to-map correlation, ICP-based matching
  - Multi-sensor fusion
- Once a loop closure is detected a constraint is introduced in the map / trajectory optimization

SLAM

Loop closure & Map / Trajectory optimization



SLAM

Loop closure & Map / Trajectory optimization

- Mathematically:

$$x_k \approx x_j \oplus \Delta x_{kj} + w_{kj}$$

where Δx_{kj} is the relative pose between x_j and x_k and $w_{kj} \sim \mathcal{N}(0, Q_{kj})$ is the uncertainty

or using graph-based optimization:

$$X^* = \arg\min_X \sum_{(i,j)\in \mathrm{E}} ||(x_i \oplus \Delta x_{ij}) \ominus x_j||^2_{\Sigma_{ij}}$$

$\oplus$ and $\ominus$ are the pose composition and pose difference operators in $SE(2)$ or $SE(3)$

$$x_j = x_i \oplus \Delta x_{ij} = T_i T_{ij}$$
$$\Delta x_{ij} = x_j \ominus x_i = T_i^{-1} T_j$$

with T_i the matrix form of the pose x_i

SLAM

Loop closure & Map / Trajectory optimization

- Trajectory and landmarks are interdependent, so the map must be updated during optimization
  - Pose adjustment causes landmarks positions to shift, as relative poses are observed during the robot's trajectory
  - The map must reflect the updated landmarks positions to maintain global consistency
- Cost of map update
  - Usually computationally expensive

    *Laserscan with occupancy grid map update requires re-rasterization and merging of **all** existing scans*
  - In practice, map updates are performed by keyframes

    Pose/observation pairs that are close / similar enough to be matched but distant enough to reduce computational overhead

SLAM

Failure modes

- SLAM can fails in some cases
  - Repetitive or ambiguous environments

    Corridors, long walls, etc.
    - Causes wrong loop closures, large global distortions
    - Often requires additional sensors to remove ambiguity
  - Textureless or specular surfaces (Visual SLAM)
    - Causes tracking loss, feature dropouts
    - Can be fixed by switching to direct method or by adding lidar/depth
  - Motion blur (Visual SLAM)
    - Causes tracking failure, large pose jumps
    - Can be fixed with increased shutter speed / reduced exposure

SLAM

Failure modes

- SLAM can fails in some cases
  - Fast rotations
    - Causes distortions (lidar), tracking failure, large pose jumps
    - Often requires fusing IMU data for interframe motion estimation
  - Dynamic environments
    - Moving objects may cause wrong data associations
    - Can be fixed with filtering or robust estimators